



RESUMO DE SISTEMAS OPERACIONAIS: SISTEMAS DE ARQUIVOS

Baseado no Cap. 4 de Tanenbaum & Bos

1. VISÃO GERAL E ABSTRAÇÃO

1.1. O Problema do Armazenamento

Para que aplicações funcionem, elas precisam armazenar e recuperar informações. A memória principal (RAM) é volátil e limitada. O armazenamento de longo prazo requer três requisitos essenciais:

1. Armazenar **grande quantidade** de dados.
2. As informações devem **sobreviver** ao término do processo (persistência).
3. **Múltiplos processos** devem poder acessar as informações (concorrência).

1.2. A Abstração "Arquivo"

O Sistema Operacional (SO) abstrai a complexidade do disco (setores, trilhas, cilindros) criando uma unidade lógica: o **Arquivo**.

- **Definição:** Unidade lógica de informação criada por processos.
- **Analogia:** Assim como o "processo" é a abstração da CPU e o "espaço de endereçamento" é a abstração da RAM, o "arquivo" é a abstração do disco.

2. ARQUIVOS (Visão do Usuário)

2.1. Nomeação de Arquivos

- **Regras:** Variam por sistema (ex: 1 a 8 caracteres no MS-DOS original, até 255 em sistemas modernos).
- **Case Sensitivity (Maiúsculas/Minúsculas):**
 - *UNIX/Linux:* Diferencia (**maria**, **Maria**, **MARIA** são arquivos diferentes).
 - *Windows/MS-DOS:* Não diferencia (todos referem-se ao mesmo arquivo).
- **Extensões:**
 - *Convenção:* Indicam o tipo de arquivo (.c, .txt, .pdf).
 - *Windows:* O SO associa a extensão a um programa para execução automática.
 - *UNIX:* Geralmente é apenas convenção, não imposta estritamente pelo SO (exceto compiladores que exigem .c, etc.).

2.2. Estrutura de Arquivos

Existem três formas principais de estruturar um arquivo internamente:

1. **Sequência de Bytes (Byte stream):** O arquivo é uma série não estruturada de bytes. O SO não sabe o que tem dentro; o significado é dado pelo programa do usuário. (Usado por **UNIX** e **Windows**). Oferece máxima flexibilidade.

2. **Sequência de Registros:** Arquivo dividido em registros de tamanho fixo. Operações de leitura/escrita são feitas em unidades de registros. (Comum em mainframes antigos).
3. **Árvore de Registros:** Registros de tamanhos variados, organizados em uma árvore B ou similar, acessados por uma chave. (Comum em grandes mainframes para processamento comercial).

2.3. Tipos de Arquivos

- **Arquivos Regulares:** Contêm informações do usuário (texto ou binário).
 - *ASCII:* Texto legível, linhas terminadas por CR e/ou LF. Editáveis.
 - *Binários:* Estrutura interna conhecida apenas pelo programa que o criou (ex: executáveis, imagens). Executáveis possuem um "número mágico" no cabeçalho para identificação pelo SO.
- **Diretórios:** Arquivos de sistema para manter a estrutura do sistema de arquivos.
- **Arquivos Especiais de Caractere:** Modelam dispositivos de E/S seriais (terminais, impressoras, redes).
- **Arquivos Especiais de Bloco:** Modelam discos.

2.4. Acesso aos Arquivos

1. **Acesso Sequencial:** Ler bytes/registros em ordem, do início ao fim. (Herança das fitas magnéticas).
2. **Acesso Aleatório (Random Access):** Ler bytes/registros fora de ordem ou pela chave. Essencial para bancos de dados.
 - *Implementação:* Operação **seek** (reposiciona o ponteiro do arquivo) seguida de **read**.

2.5. Atributos (Metadados)

Informações adicionais associadas ao arquivo, não os dados em si. Exemplos:

- **Proteção:** Quem pode acessar (leitura/escrita/execução).
- **Flags:** Oculto, Sistema, Somente Leitura, Arquivamento (para backup), Temporário.
- **Timestamps:** Data de criação, último acesso, última modificação.
- **Tamanho:** Tamanho atual do arquivo.

2.6. Operações com Arquivos (Chamadas de Sistema)

- **Create:** Cria arquivo sem dados.
- **Delete:** Remove o arquivo.
- **Open:** Traz atributos e lista de endereços de disco para a memória (prepara para acesso rápido). Retorna um descritor (handle).
- **Close:** Libera recursos da tabela interna. Força escrita de blocos finais.
- **Read / Write:** Ler/Escriver dados na posição atual.
- **Append:** Adicionar dados ao final.
- **Seek:** Reposicionar o ponteiro de arquivo (acesso aleatório).
- **Get/Set Attributes:** Ler ou modificar metadados.
- **Rename:** Mudar o nome.

3. DIRETÓRIOS

3.1. Organização

- **Nível Único:** Um único diretório para todos os arquivos do sistema. (Simples, mas impossível para muitos usuários/arquivos).
- **Hierárquico (Árvore):** Usuários podem criar subdiretórios arbitrariamente. Padrão moderno.

3.2. Nomes de Caminhos (Pathnames)

- **Caminho Absoluto:** Começa no diretório raiz.
 - Windows: `\usr\ast\mailbox`
 - UNIX: `/usr/ast/mailbox`
- **Caminho Relativo:** Começa no **diretório de trabalho atual**.
 - Ex: Se estou em `/usr`, o caminho relativo é `ast/mailbox`.
- **Entradas Especiais:**
 - `.` (ponto): Diretório atual.
 - `..` (ponto-ponto): Diretório pai.

3.3. Operações de Diretório

- **Create / Delete:** Criar/remover diretórios (geralmente só remove se estiver vazio).
 - **Opendir / Closedir / Readdir:** Para ler o conteúdo de um diretório sem depender da estrutura interna.
 - **Link:** Cria uma ligação (o arquivo aparece em mais de um diretório).
 - **Unlink:** Remove uma entrada de diretório. Se for a última ligação, o arquivo é deletado.
-

4. IMPLEMENTAÇÃO DO SISTEMA DE ARQUIVOS (Tópico Crítico)

O objetivo é mapear arquivos lógicos em blocos físicos no disco.

4.1. Layout do Disco

- **MBR (Master Boot Record):** Setor 0. Contém o código de inicialização e a **Tabela de Partição**.
- **Partição:** Divisão lógica do disco. Contém:
 1. **Bloco de Inicialização (Boot block):** Carrega o SO.
 2. **Superbloco:** Parâmetros chave do sistema de arquivos (tipo, tamanho, qtd blocos).
 3. **Gerenciamento de Espaço Livre:** (Bitmap ou Lista).
 4. **I-nodes:** (Em sistemas tipo UNIX).
 5. **Diretório Raiz.**
 6. **Arquivos e Diretórios (Dados).**

4.2. Implementação de Arquivos (Alocação de Blocos)

A. Alocação Contígua

- **Como funciona:** Armazena o arquivo em blocos consecutivos no disco.
- **Vantagens:** Implementação simples (só guarda endereço inicial e tamanho), leitura excelente (uma busca + transferência rápida).
- **Desvantagens:** **Fragmentação Externa** (disco fica cheio de buracos). Necessidade de saber tamanho final do arquivo na criação.
- **Uso atual:** CD-ROMs, DVDs (mídia somente leitura).

B. Alocação por Lista Encadeada

- **Como funciona:** Cada bloco contém um ponteiro para o próximo bloco.
- **Vantagens:** Sem fragmentação externa.
- **Desvantagens:** Acesso aleatório muito lento (tem que percorrer a lista). O tamanho do bloco de dados não é mais uma potência de 2 (ponteiro ocupa espaço).

C. Lista Encadeada com Tabela na Memória (FAT)

- **Como funciona:** A lista de ponteiros é retirada dos blocos físicos e colocada numa tabela na RAM chamada **FAT (File Allocation Table)**.
- **Vantagens:** O bloco inteiro é usado para dados. Acesso aleatório mais rápido (percorre a lista na RAM, não no disco).
- **Desvantagens:** A tabela precisa estar inteira na memória. Para discos grandes, a FAT consome muita RAM (inviável para discos de TBs com blocos pequenos).
- **Exemplo:** MS-DOS, Windows (FAT16, FAT32).

D. I-nodes (Index-nodes)

- **Como funciona:** Cada arquivo tem uma pequena estrutura de dados chamada **i-node** que lista atributos e endereços de disco dos blocos.
- **Estrutura do I-node:**
 - Atributos (dono, tamanho, datas).
 - Endereços Diretos (apontam para dados).
 - Endereço Indireto Simples (aponta para bloco de ponteiros).
 - Endereço Indireto Duplo (aponta para bloco de ponteiros para ponteiros).
- **Vantagens:** O i-node só precisa estar na memória quando o arquivo está aberto. Economiza RAM comparado à FAT.
- **Exemplo:** UNIX.

4.3. Implementação de Diretórios

A função do diretório é mapear o **Nome ASCII** para a **Informação de localização** (I-node ou endereço).

1. **Lista Simples:** Lista de entradas (Nome, Atributos, Endereços). Busca linear lenta.
2. **Referência a I-node:** Entrada contém apenas (Nome, Número do I-node). Os atributos ficam no I-node.
3. **Nomes Longos:**
 - Opção A: Reservar espaço fixo (ex: 255 bytes) - desperdício.
 - Opção B: Entradas de tamanho variável ou Heap no final do diretório.
4. **Aceleração de Busca:** Uso de **Tabelas de Hash** em vez de busca linear para diretórios muito grandes.

4.4. Arquivos Compartilhados

- **Hard Link (Ligação Estrita):**
 - Duas entradas de diretório apontam para o **mesmo i-node**.
 - Incrementa um contador no i-node.

- Arquivo só é deletado quando contador = 0.
- Problema: Não funciona entre partições diferentes.
- **Symbolic Link (Ligação Simbólica):**
 - Um arquivo pequeno que contém o **caminho** (path) de outro arquivo.
 - Vantagem: Funciona entre discos/redes.
 - Desvantagem: Mais lento (overhead de leitura extra). Se o original for movido, o link quebra.

4.5. Sistemas de Arquivos Estruturados em Log (LFS)

- **Conceito:** Otimizar para escritas. O disco é tratado como um log circular.
- **Funcionamento:** Agrupa todas as escritas pendentes (i-nodes, dados, diretórios) em um grande segmento e escreve contiguamente no fim do log. Transforma escritas aleatórias em sequenciais.
- **Limpeza:** Um thread "limpador" compacta o log para liberar espaço.

4.6. Journaling (Sistemas de Arquivos com Diário)

- **Problema:** Quedas de energia durante operações complexas (ex: remover arquivo requer 3 passos) deixam o disco inconsistente.
- **Solução:** Manter um **diário (journal)**.
 1. Escreve no diário o que *vai* fazer.
 2. Executa as operações reais.
 3. Apaga a entrada no diário.
- **Recuperação:** Se o sistema cair, ao reiniciar, o SO lê o diário e completa/refaz as operações pendentes.
- **Idempotência:** As operações devem ser repetíveis sem erro.
- **Exemplos:** NTFS, ext3, ext4, ReiserFS.

4.7. Sistemas de Arquivos Virtuais (VFS)

- **Objetivo:** Permitir que múltiplos sistemas de arquivos (ext2, NTFS, ISO9660) coexistam transparentemente.
- **Funcionamento:** O SO define uma interface abstrata (VFS). As chamadas do usuário (open, read) vão para o VFS, que chama o driver específico do sistema de arquivos concreto.
- **Estrutura:** Usa **v-nodes** (nós virtuais) na memória para representar arquivos abertos, independentemente do FS de origem.

5. GERENCIAMENTO E OTIMIZAÇÃO

5.1. Gerenciamento de Espaço em Disco

1. **Tamanho do Bloco:**
 - *Blocos Grandes:* Alta taxa de transferência, mas muito desperdício de espaço (**fragmentação interna**) em arquivos pequenos.
 - *Blocos Pequenos:* Boa eficiência de espaço, mas baixo desempenho (muitas buscas).
 - *Média:* 4 KB é um compromisso comum.
2. **Gerenciamento de Blocos Livres:**
 - *Lista Encadeada:* Blocos livres apontam para outros blocos livres. Bom quando disco está cheio.

- *Bitmap (Mapa de Bits)*: 1 bit por bloco (1=livre, 0=ocupado). Mais fácil encontrar blocos contíguos.

3. **Cotas de Disco**: Tabelas que rastreiam uso por usuário. Limite "soft" (aviso) e "hard" (bloqueio).

5.2. Backups

- **Cópia Física**: Copia bloco a bloco (rápido, simples, mas copia blocos vazios e não permite pular arquivos).
- **Cópia Lógica**: Percorre a árvore de diretórios (mais lento, mas flexível).
- **Tipos**: Completo vs. Incremental (apenas o que mudou).

5.3. Consistência do Sistema de Arquivos

Utilitários como **fsck** (UNIX) ou **sfc** (Windows) verificam erros pós-crash.

1. **Verificação de Blocos**: Cria tabelas de contadores. Verifica se blocos estão marcados como livres e ocupados simultaneamente (erro grave).
2. **Verificação de Arquivos**: Compara contagem de links nos i-nodes com o número real de entradas de diretório.

5.4. Desempenho do Sistema de Arquivos

O acesso a disco é lento. Otimizações são essenciais:

1. **Caching (Cache de Blocos/Buffer)**:
 - Manter blocos usados recentemente na RAM.
 - Usa tabelas de hash para busca rápida.
 - *Write-through*: Escreve no disco imediatamente (seguro).
 - *Delayed-write*: Escreve na cache e sincroniza periodicamente (rápido, risco de perda de dados - **sync** no UNIX).
2. **Leitura Antecipada (Read-ahead)**:
 - Ler o próximo bloco antes que seja solicitado (ótimo para acesso sequencial).
3. **Redução do Movimento do Braço**:
 - Colocar blocos do mesmo arquivo próximos.
 - Agrupar i-nodes em **grupos de cilindros** espalhados pelo disco (não tudo no início).

5.5. Desfragmentação

Mover arquivos para torná-los contíguos novamente. Útil em sistemas Windows/FAT. Menos crítico em Linux/ext ou em SSDs (onde desfragmentar desgasta o disco sem ganho de performance).

6. EXEMPLOS DE SISTEMAS DE ARQUIVOS

6.1. MS-DOS (FAT)

- Entrada de diretório: 32 bytes fixos (Nome 8+3, Atributos, Tempo, Primeiro Bloco, Tamanho).
- **FAT (12, 16, 32)**: O número indica quantos bits por endereço de disco.
- **FAT-32**: Permite discos de até 2 TB (teoricamente) com blocos de 4 KB. Usa endereços de 28 bits.
- Não usa i-nodes; usa a tabela FAT na RAM para encadeamento.

6.2. UNIX V7

- Usa **I-nodes**.
- Entrada de diretório simples: 2 bytes para nº do i-node + 14 bytes para nome.
- Suporta blocos Indiretos (Simples, Duplos e Triplos) para arquivos grandes.
- Busca de arquivo: `/usr/ast/mbox`.
 1. Busca `/` (raiz, local fixo).
 2. Lê diretório raiz, acha i-node de `usr`.
 3. Lê bloco de dados de `usr`, acha i-node de `ast`.
 4. Lê bloco de dados de `ast`, acha i-node de `mbox`.

6.3. CD-ROM (ISO 9660)

- Projetado para mídia "Write-once". Arquivos contíguos.
- **Níveis:**
 - Nível 1: Nomes 8+3, contíguos.
 - Nível 2: Nomes até 31 caracteres.
 - Nível 3: Arquivos não contíguos (extensões).
- **Extensões:**
 - **Rock Ridge:** Adiciona semântica UNIX (permissões rwx, nomes longos, links simbólicos) usando campos extras não definidos pelo ISO 9660.
 - **Joliet:** Extensão da Microsoft. Suporta Unicode e nomes longos.



NOTAS PARA QUESTÕES DA LISTA AVALIATIVA

- **Nomes longos em diretórios:** Em diretórios de tamanho fixo (MS-DOS antigo), não cabem. A solução moderna (ex: ext2/3/4 ou FAT com LFN) é usar entradas de tamanho variável ou um "heap" no final do diretório para armazenar a string do nome, mantendo um ponteiro na entrada fixa.
- **Journaling:** Essencial explicar o conceito de *idempotência* e a sequência: Escrever no Log `$\to$` Comitar `$\to$` Executar `$\to$` Limpar Log.
- **I-nodes e Alocação:** Lembre-se dos ponteiros diretos (para arquivos pequenos) e indiretos (para arquivos gigantes). O i-node precisa estar na memória apenas quando o arquivo está aberto.
- **FAT:** A tabela mapeia cada bloco do disco. O valor na tabela aponta para o próximo bloco do arquivo. Fim de arquivo = valor especial. Tabela inteira deve residir na RAM.
- **ACLs (Listas de Controle de Acesso):** Enquanto o UNIX tradicional usa bits rwx (Dono, Grupo, Outros), as ACLs permitem especificar permissões detalhadas para *usuários específicos* (Ex: "O usuário Bob pode ler, Alice não pode nada, Grupo Dev pode escrever"). Elas estendem o modelo de atributos.
- **Programação (statvfs, readv):**
 - **statvfs:** Chamada de sistema POSIX para obter estatísticas do sistema de arquivos (espaço livre, total, nº de i-nodes).
 - **readv:** Leitura "scatter-gather". Lê dados de um arquivo e os espalha em múltiplos buffers não contíguos na memória em uma única chamada atômica.